

Lecture 2

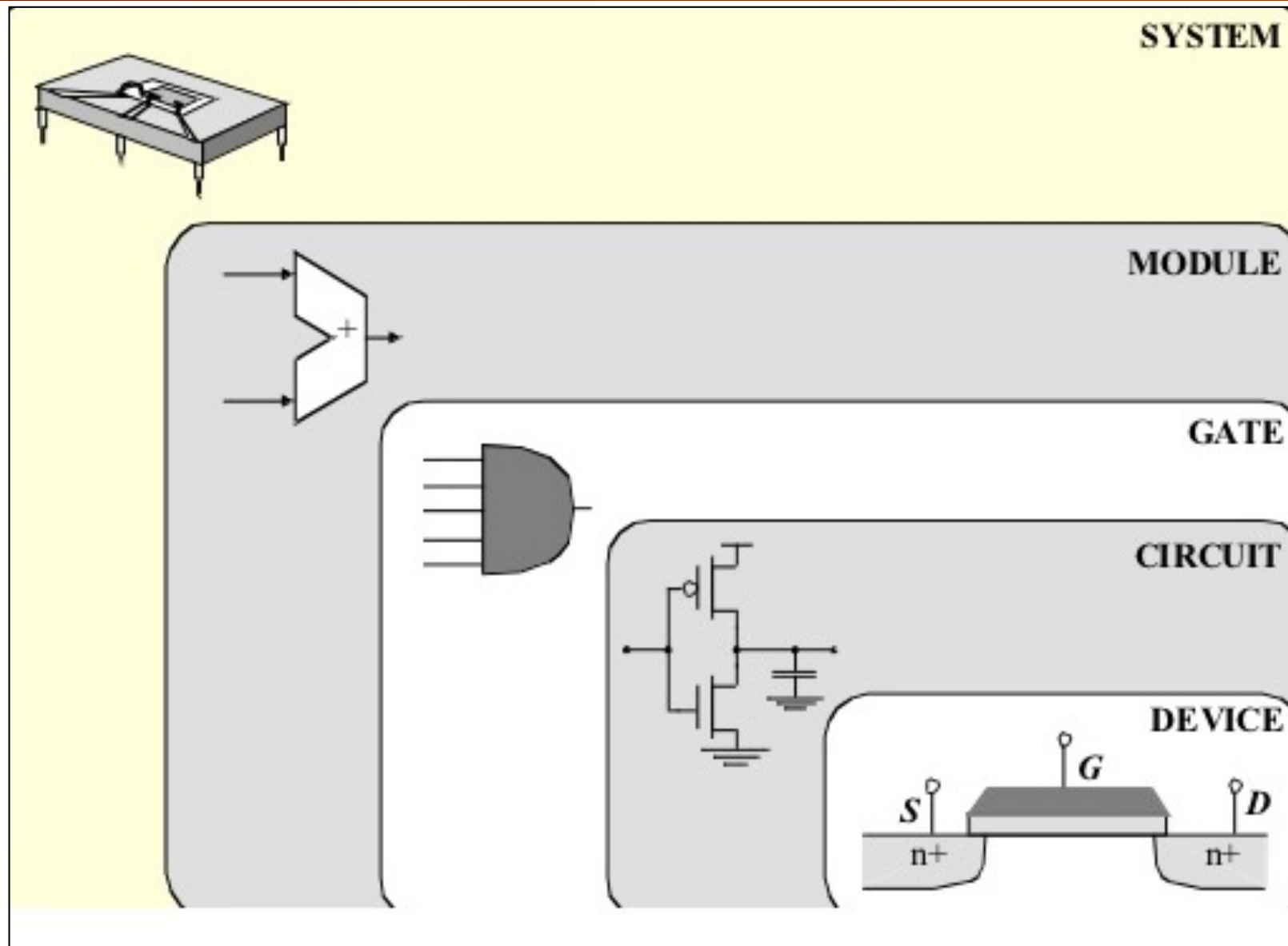
VLSI Design Flow overview

Prof Peter YK Cheung

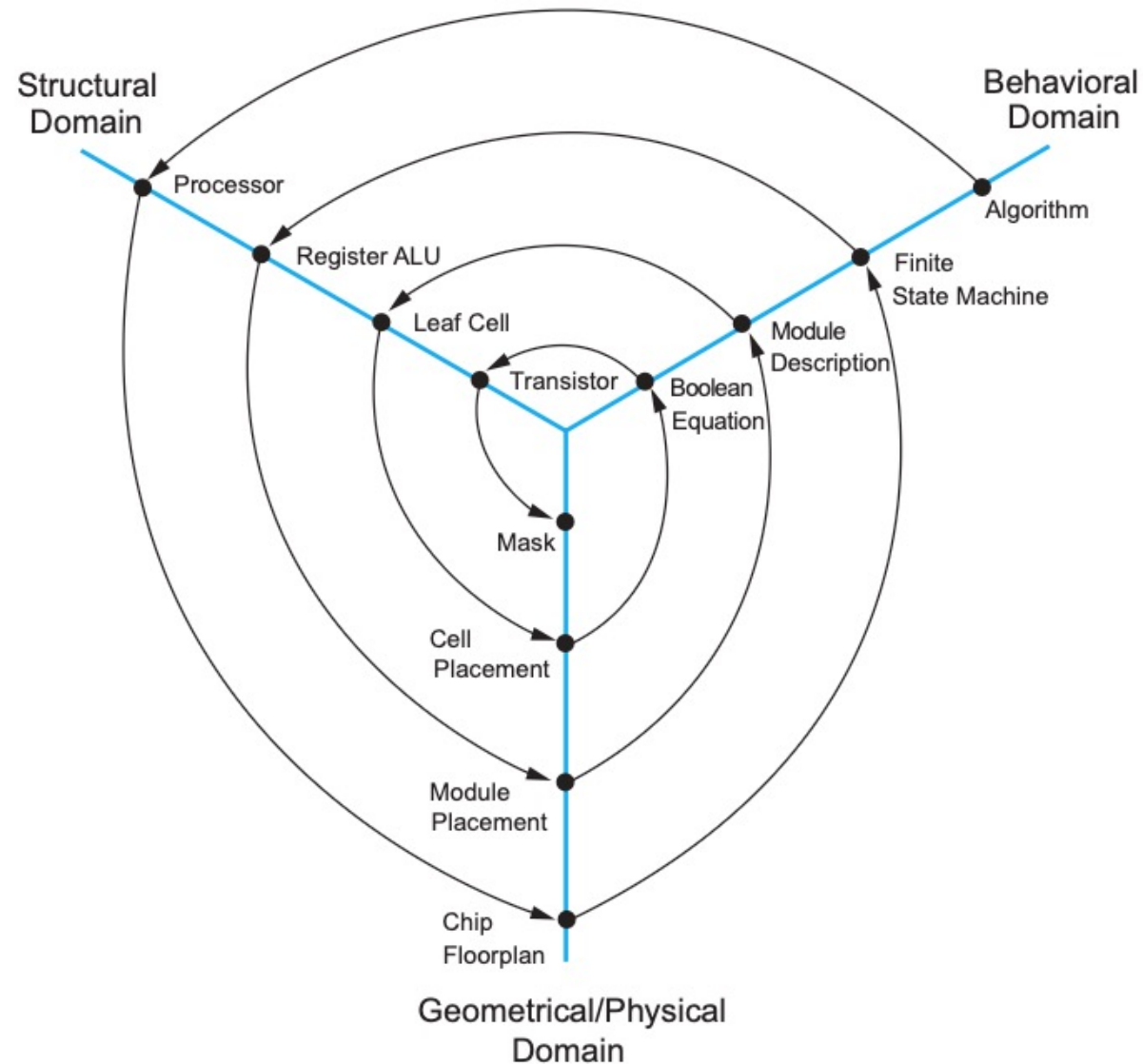
Department of Electrical & Electronic Engineering

URL: www.ee.ic.ac.uk/pcheung/teaching/EE4-VLSI/
E-mail: p.cheung@imperial.ac.uk

Levels of Abstraction in Digital Design



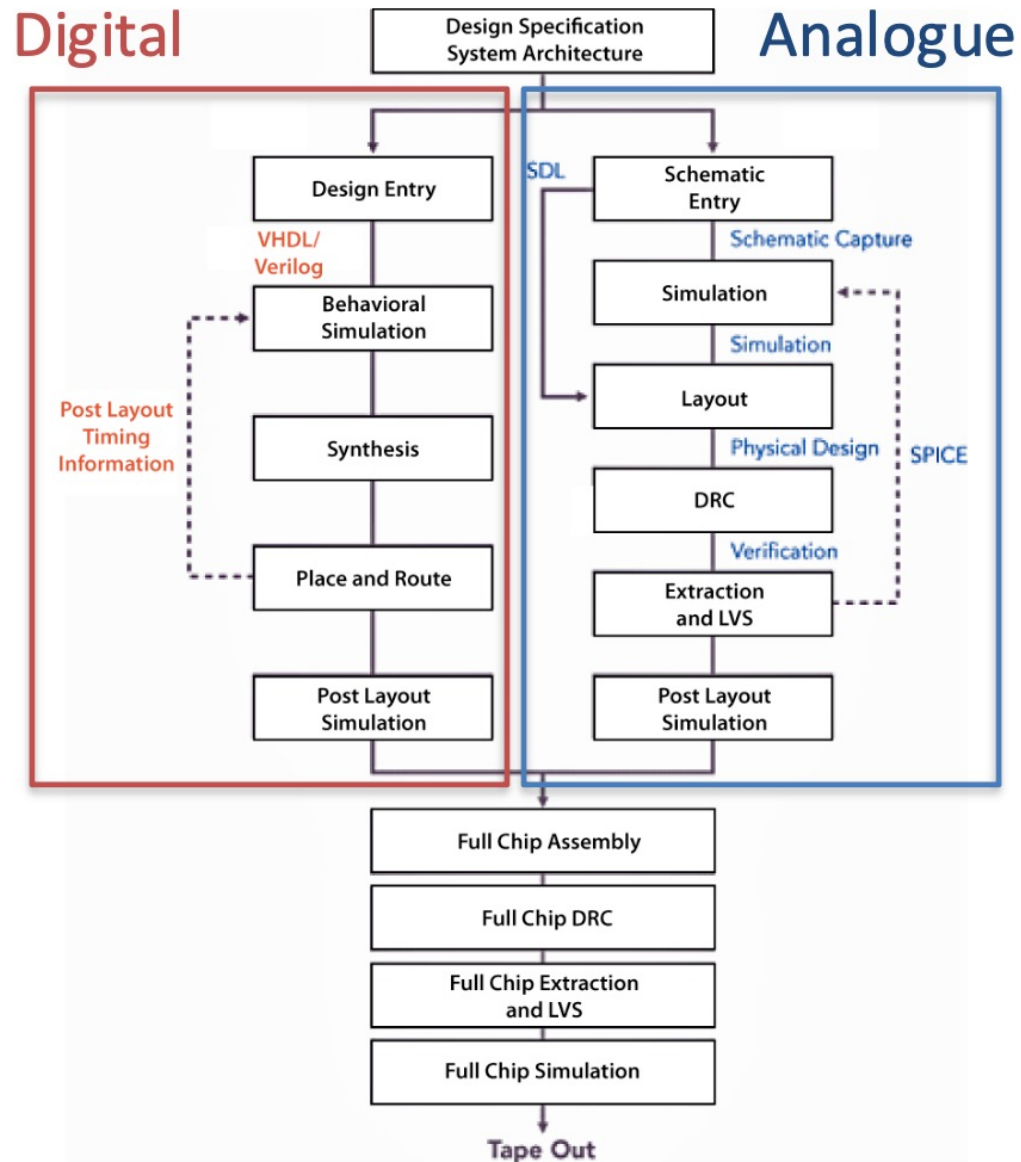
Levels of Abstraction in Digital Design



Source: Gajski 1983 & Kang 2003

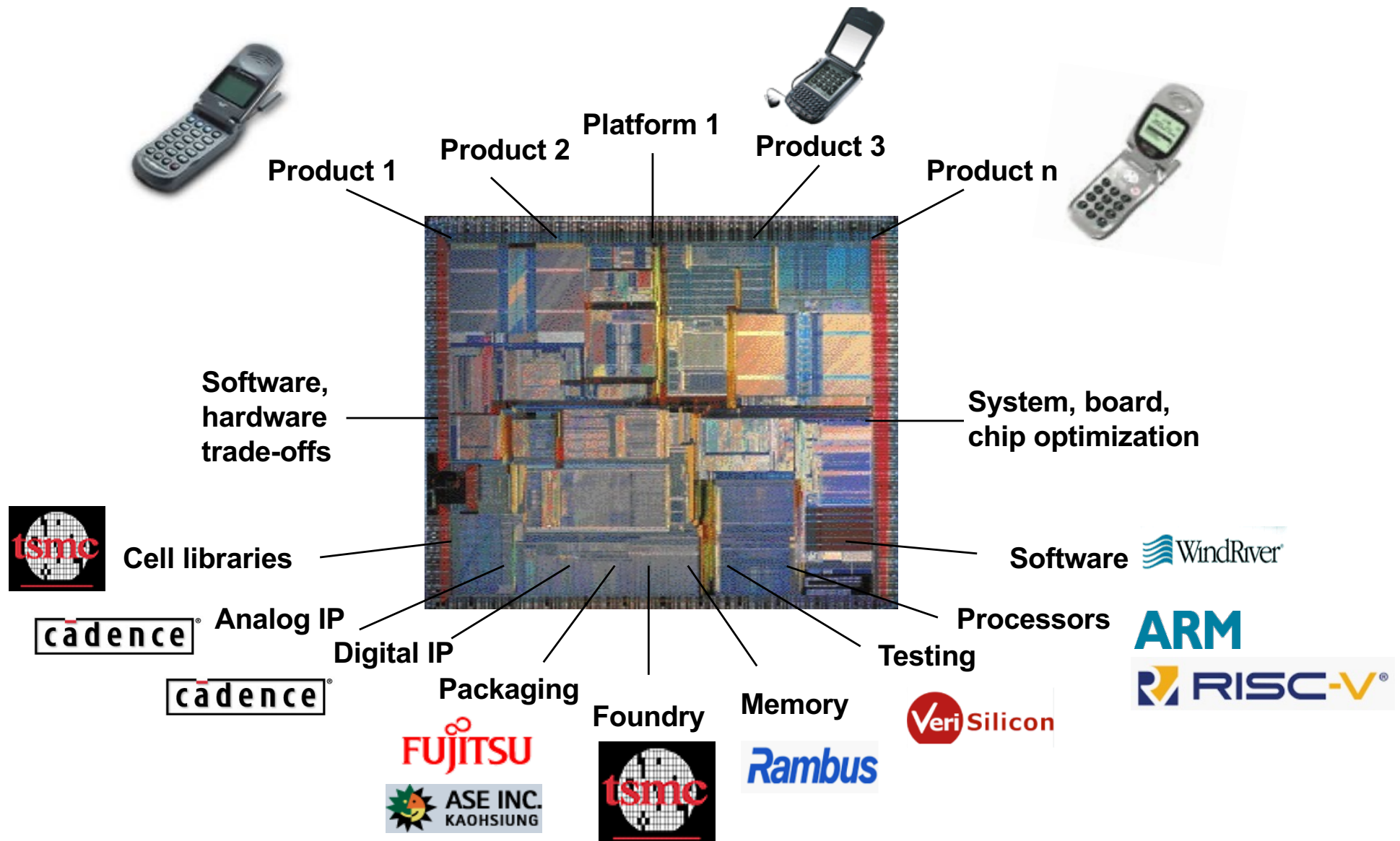


Mixed signal VLSI Design Flow



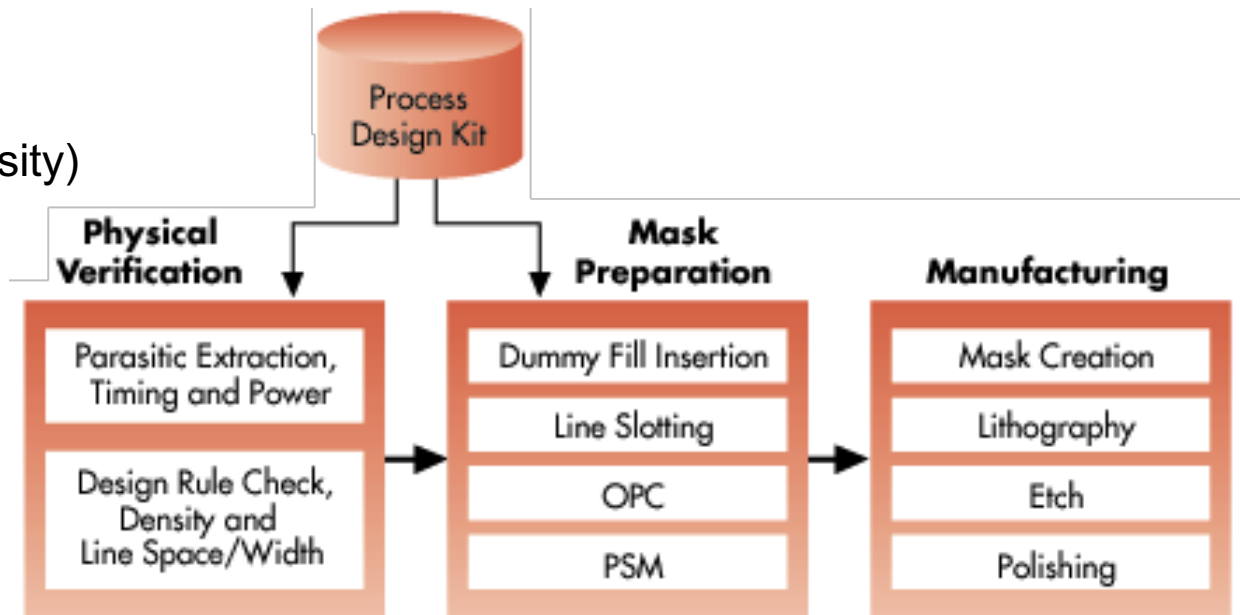
Source: Notes from T Constandinou

IP based VLSI Design

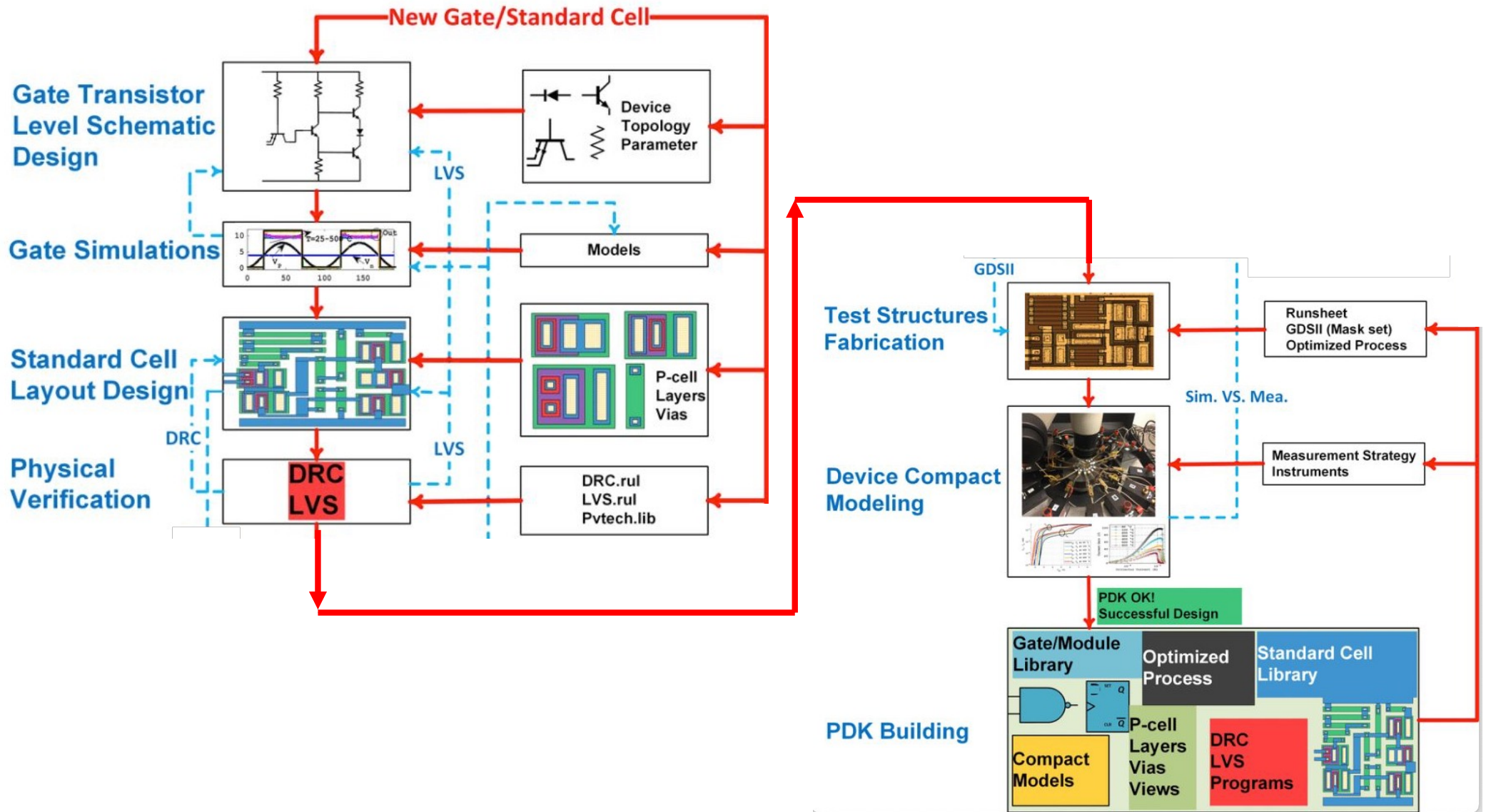


What is a Process Design Kit (PDK)

- ◆ Link between chip maker (foundry) and EDA tool supplier (TSMC → Cadence)
- ◆ PDK is for a specific process (we use TSMC 65nm low power process)
- ◆ PDK consists for set of files in standardised format for different design steps:
 - Primitive devices (e.g. transistors, capacitor etc.)
 - Device parameters
 - Device symbol
 - Technology related information
 - Layers and layer names
 - Appearance (colour)
 - Constraints (e.g. current density)
 - Verification information
 - Design rule checking (DRC)
 - Layout vs Schematic (LVS)
 - Device extraction
 - Simulation models
 - Standard cell libraries
 - Documentation

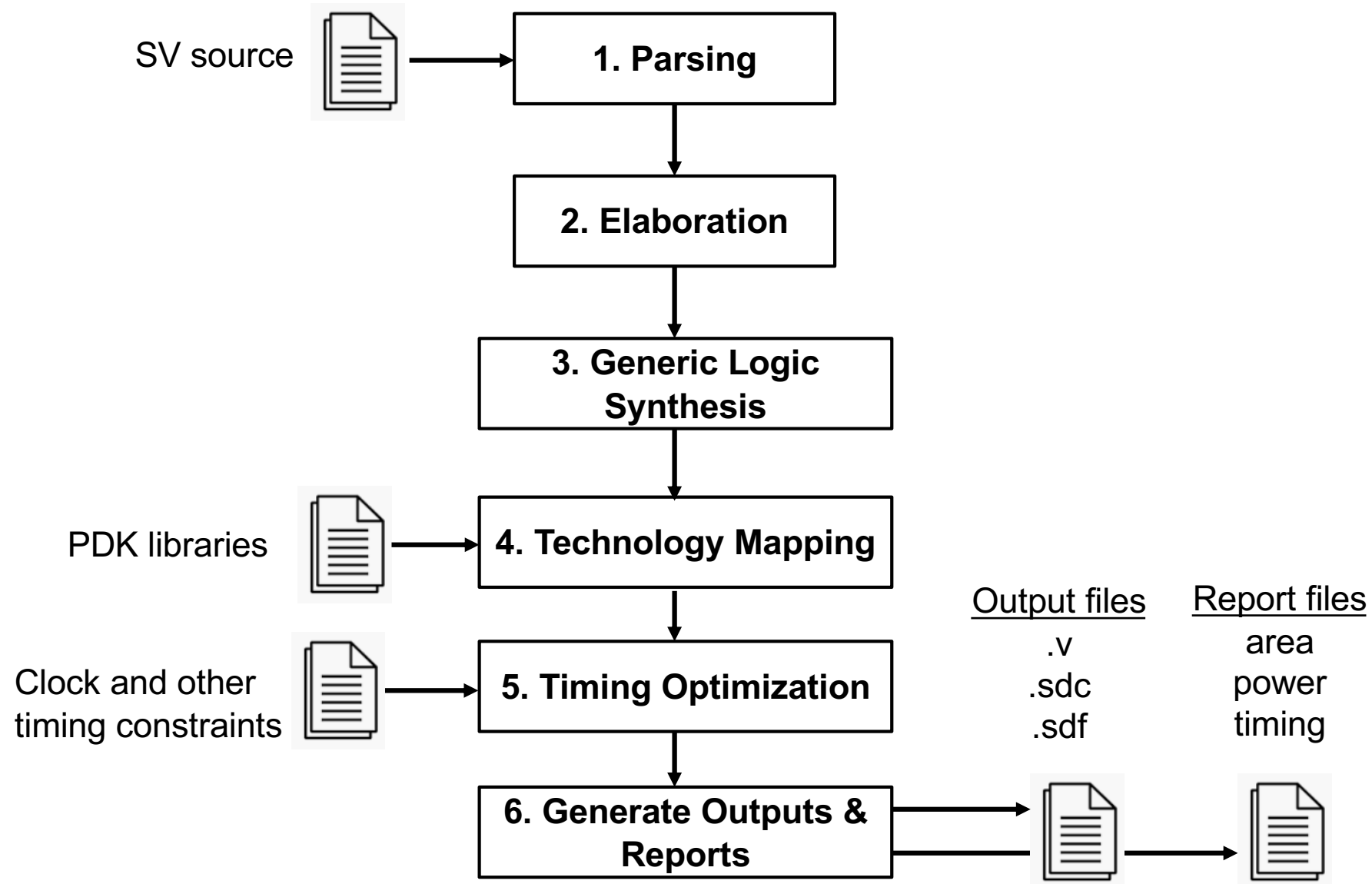


PDK Build Process



Source: Gupta

Genus - RTL to Standard Cell Synthesis



Synthesize LFSR4

```

module lfsr4 (
    input logic      clk,      // clock
    input logic      rst,      // reset
    output logic [3:0] data_out // pseud
);

logic [4:1]      sreg;

always_ff @ (posedge clk, posedge rst)
    if (rst)
        sreg <= 4'b1;
    else

```

RTL source

```

module lfsr4(clk, rst, data_out);
    .....
    wire n_0, n_1;
    DFSNQD1BWP7T \sreg_reg[1] (.SDN (n_0), .CP (clk
        | (data_out[0]));
    CKXOR2D0BWP7T g16__8780(.A1 (data_out[2]), .A2
        | (n_1));
    DFCNQD1BWP7T \sreg_reg[4] (.CDN (n_0), .CP (clk
        | .Q (data_out[3]));
    DFCNQD1BWP7T \sreg_reg[3] (.CDN (n_0), .CP (clk
        | .Q (data_out[2]));
    DFCNQD1BWP7T \sreg_reg[2] (.CDN (n_0), .CP (clk
        | .Q (data_out[1]));
    INVDD0BWP7T g20(.I (rst), .ZN (n_0));
endmodule

```

Synthesized circuit

Genus

```

# Variable setup
set _HDL_DIRECTORY ./SRC
set HDL_FILES [list lfsr4.
set DESIGN lfsr4

# Clock name should match
set CLOCK_NAME clk
set CLOCK_PERIOD_ps 1000

set GEN_EFF medium
set MAP_OPT_EFF high

set _OUTPUTS_PATH OUTPUTS
set _REPORTS_PATH REPORTS

```

Tcl commands/script

Instance	Module	Cell Count	Cell Area	Net Area	Total Area
lfsr4		6	30.520	5.771	36.291

Area report

Synthesis Steps (preparation) (1)

```
# Variable setup
set _HDL_DIRECTORY ./SRC
set HDL_FILES [list lfsr4.]
set DESIGN lfsr4
```

```
# Clock name should match
set CLOCK_NAME clk
set CLOCK_PERIOD_ps 1000
```

```
set GEN_EFF medium
set MAP_OPT_EFF
```

```
set _OUTPUTS_PA
set _REPORTS_PA
```

```
set_db init_hdl_search_path $_HDL_DIRECTORY;
set_db library "/usr/local/cadence/kits/tsmc/be
set_db lef_library "/usr/local/cadence/kits/tsm
```

```
set_db
set_db
set_db
```

```
# Load the design and check basic syntax
read_hdl -sv ${HDL_FILES}
elaborate ${DESIGN}
check_design -unresolved
check_design -unloaded
```

```
# Tell Genus about the clock signal
create_clock -domain domain1 -name ${CLOCK_NAME}
set_db clock:${DESIGN}/${CLOCK_NAME} .setup_unc
set_clock_uncertainty -hold [expr 0.02 * ${CLO
set_clock_uncertainty -setup [expr 0.02 * ${CLO
set_clock_transition -rise 50 ${CLOCK_NAME}
set_clock_transition -fall 50 ${CLOCK_NAME}
```

Synthesis Steps (synthesis) (2)

```
# First synthesize to generic gates
set_db syn_generic_effort $GEN_EFF
syn_generic
```

```
# Then map to TSMC standard cells
set_db syn_map_effort $MAP_OPT_EFF
```

```
# Optimize netlist
```

```
# Export Design Files
write_snapshot -directory ${_REPORTS_PATH}/final -tag final
report_summary -directory ${_REPORTS_PATH}
write_hdl > ${_REPORTS_PATH}/hdl_syn_opt.v
write_sdc > ${_REPORTS_PATH}/sdc_syn_opt.sdc
write_sdf > ${_REPORTS_PATH}/sdf_syn_opt.sdf
write_script > ${_REPORTS_PATH}/script_syn_opt.sh
write_design -hdl ${_REPORTS_PATH}/hdl_syn_opt.v -sdc ${_REPORTS_PATH}/sdc_syn_opt.sdc -sdf ${_REPORTS_PATH}/sdf_syn_opt.sdf -script ${_REPORTS_PATH}/script_syn_opt.sh
write_db -all -directory ${_REPORTS_PATH}/final -tag final
```

```
# Write reports
report_qor > ${_REPORTS_PATH}/${DESIGN}_qor.rpt
report_area > ${_REPORTS_PATH}/${DESIGN}_area.rpt
report_dp > ${_REPORTS_PATH}/${DESIGN}_datapath_incr.rpt
report_messages > ${_REPORTS_PATH}/${DESIGN}_messages.rpt
report_gates > ${_REPORTS_PATH}/${DESIGN}_gates.rpt
report_timing > ${_REPORTS_PATH}/${DESIGN}_timing.rpt
report_power > ${_REPORTS_PATH}/${DESIGN}_power.rpt
```

Genus synthesized circuit

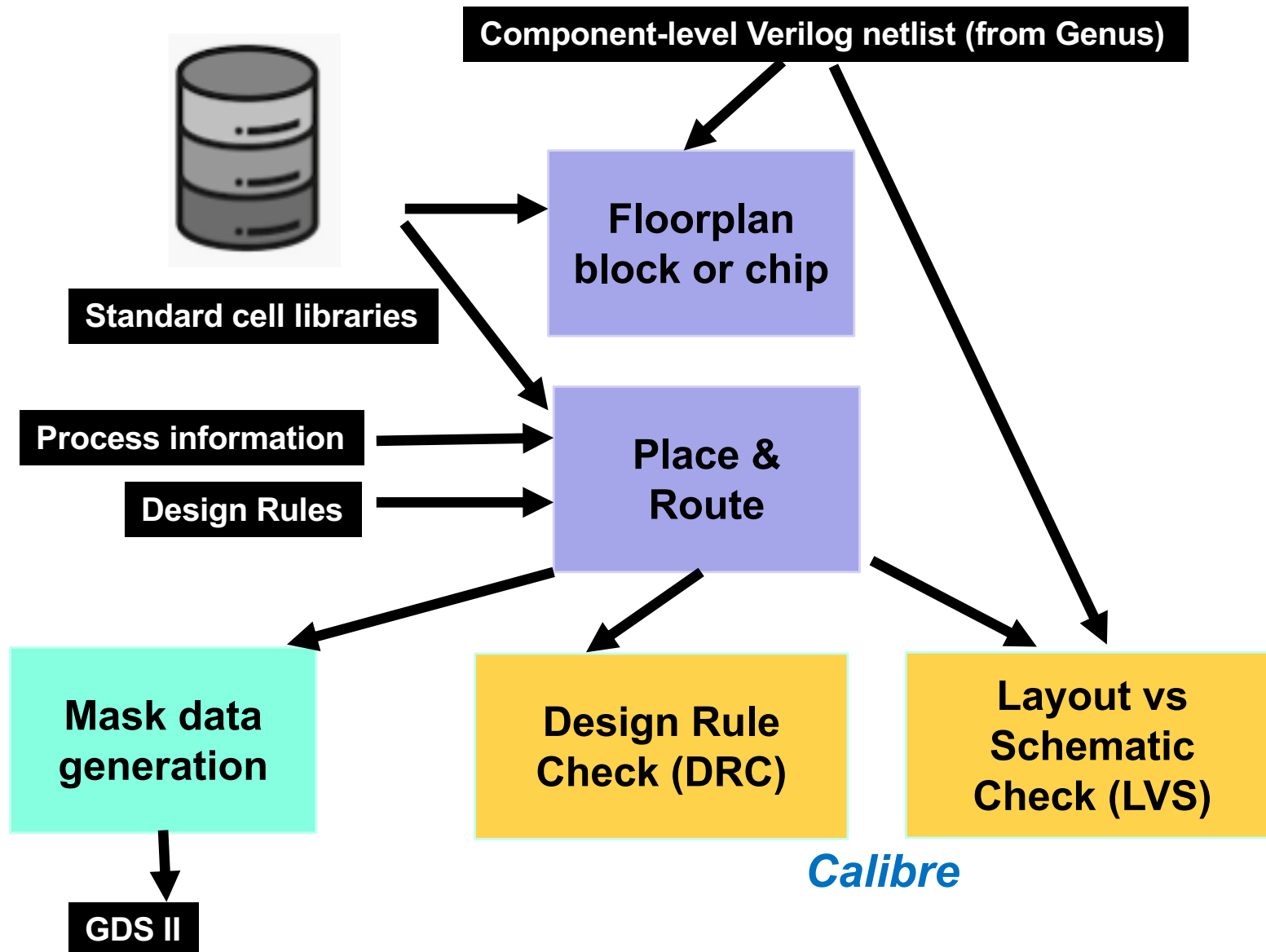
```
module lfsr4 (  
    input logic      clk,          // clock  
    input logic      rst,          // reset  
    output logic [3:0] data_out    // pseud  
);  
  
logic [4:1]      sreg;  
  
always_ff @ (posedge clk, posedge  
    if (rst)  
        sreg <= 4'b1;  
    else  
        sreg <= {sreg[3:1], sreg[3] ^  
assign data_out = sreg;  
endmodule
```

**RTL source
In SystemVerilog**

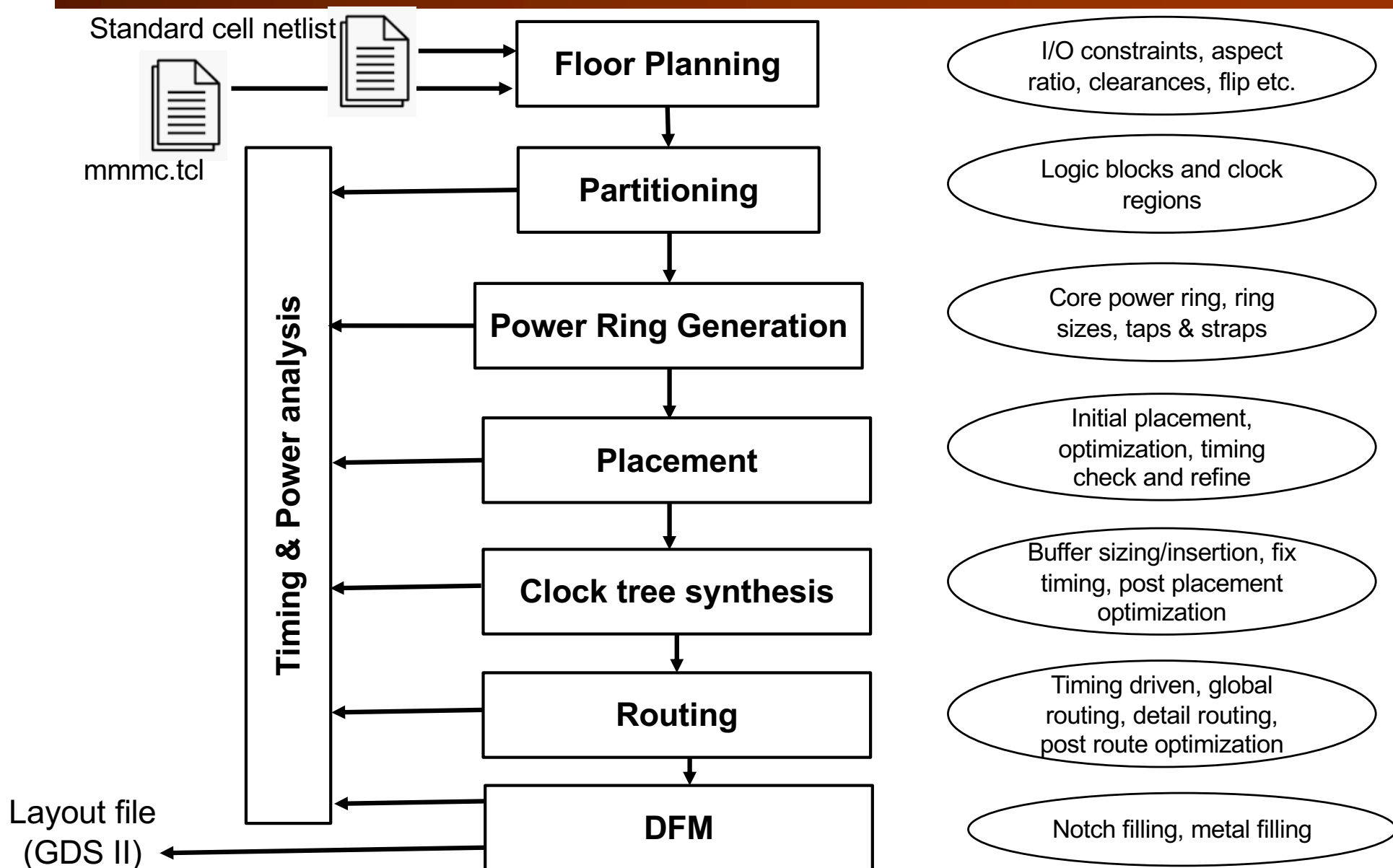
```
module lfsr4(clk, rst, data_out);  
    input clk, rst;  
    output [3:0] data_out;  
    wire clk, rst;  
    wire [3:0] data_out;  
    wire n_0, n_1;  
    DFSNQD1BWP7T \sreg_reg[1] (.SDN (n_0), .CP (clk), .D (n_1), .Q  
        (data_out[0]));  
    CKXOR2D0BWP7T g16__8780(.A1 (data_out[2]), .A2 (data_out[3]), .Z  
        (n_1));  
    DFCNQD1BWP7T \sreg_reg[4] (.CDN (n_0), .CP (clk), .D (data_out[2]),  
        .Q (data_out[3]));  
    DFCNQD1BWP7T \sreg_reg[3] (.CDN (n_0), .CP (clk), .D (data_out[1]),  
        .Q (data_out[2]));  
    DFCNQD1BWP7T \sreg_reg[2] (.CDN (n_0), .CP (clk), .D (data_out[0]),  
        .Q (data_out[1]));  
    INVDP0BWP7T g20(.I (rst), .ZN (n_0));  
endmodule
```

**synthesized netlist
in Verilog**

Innovus in context

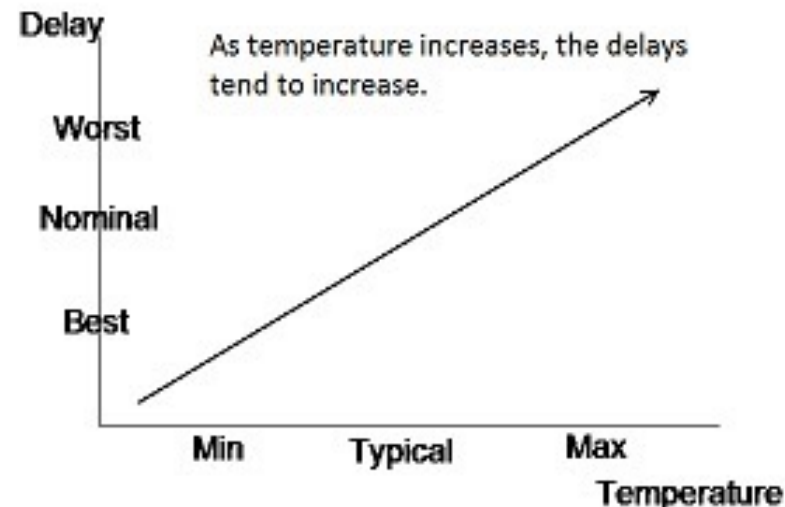
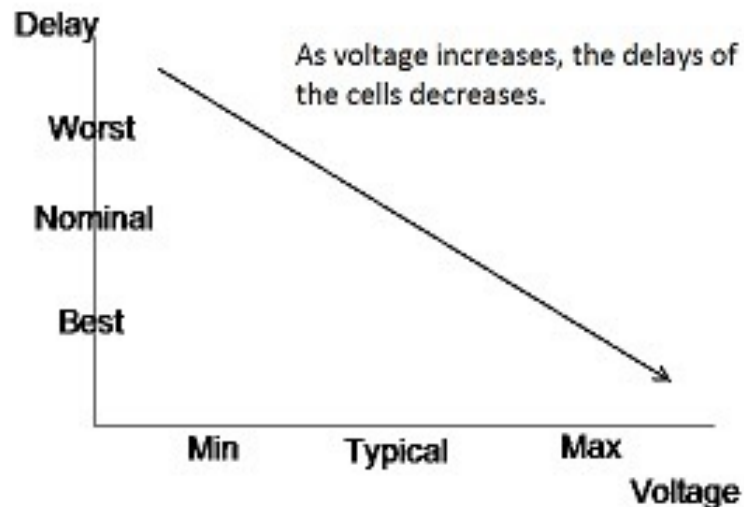
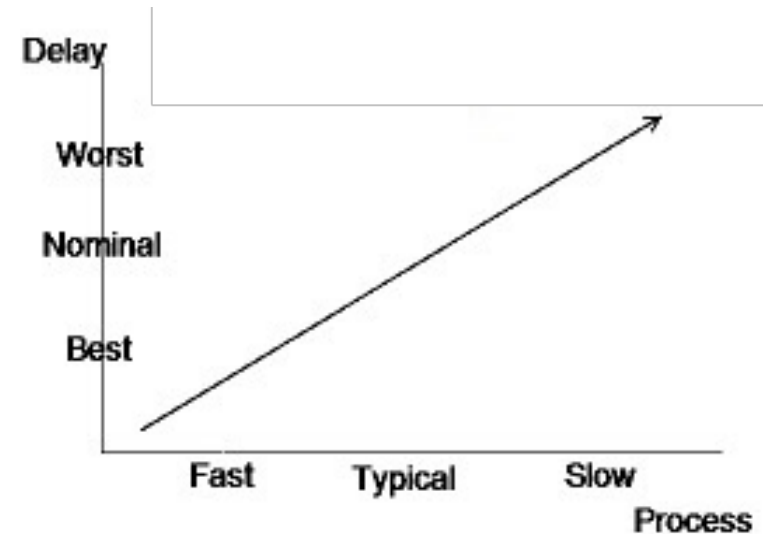


Innovus – Netlist to Layout Flow



What is PVT?

- ◆ PVT stands for Process, Voltage and Temperature
- ◆ Need to tell Place-and-Route tools these variations
- ◆ Ensure circuit works when manufactured
- ◆ Must also work under worst case environment
- ◆ Specified PVT through MMMC file (Tcl format)



Multi-Mode, Multi-Corner (MMMC)

```
#####
## MMMC Timing Constraints Setup
#####

# --- Specify Library
create_library_set -name libs_typ -timing { \
    "/usr/local/cadence/kits/tsmc/beLibs/65nm/TSMCHOME/digital/Front_End/timing_power_noise/NLDM/tcl
    "/usr/local/cadence/kits/tsmc/beLibs/65nm/TSMCHOME/digital/Front_End/timing_power_noise/NLDM/tpl
    "/usr/local/cadence/kits/tsmc/beLibs/65nm/TSMCHOME/digital/Front_End/timing_power_noise/NLDM/tpl
    "/usr/local/cadence/kits/tsmc/beLibs/65nm/TSMCHOME/digital/Front_End/timing_power_noise/NLDM/tpl
    "/usr/local/cadence/kits/tsmc/beLibs/65nm/TSMCHOME/digital/Front_End/timing_power_noise/NLDM/tpl
    "/usr/local/cadence/kits/tsmc/beLibs/65nm/TSMCHOME/digital/Front_End/timing_power_noise/NLDM/tpl
}

# --- Specify RC corners
create_rc_corner -name tsmc65_rc_corner_typ \
    -cap_table {/usr/local/cadence/kits/tsmc/beLibs/65nm/TSMCHOME/digital/Back_End/lef/tcbn65lp
    -T 25

# --- Specify delay corners
create_delay_corner -name corner_typ -library_set {libs_typ} -rc_corner {tsmc65_rc_corner_typ}

# --- Specify constraints (from SDC file)
create_constraint_mode -name CONSTRAINTS -sdc_files ${SDC_FILE}

# --- Specify analysis view (functional only for now)
create_analysis_view -name {functional_typ} -delay_corner {corner_typ} -constraint_mode {CONSTRAINTS}
set_analysis_view -setup {functional_typ} -hold {functional_typ}
```

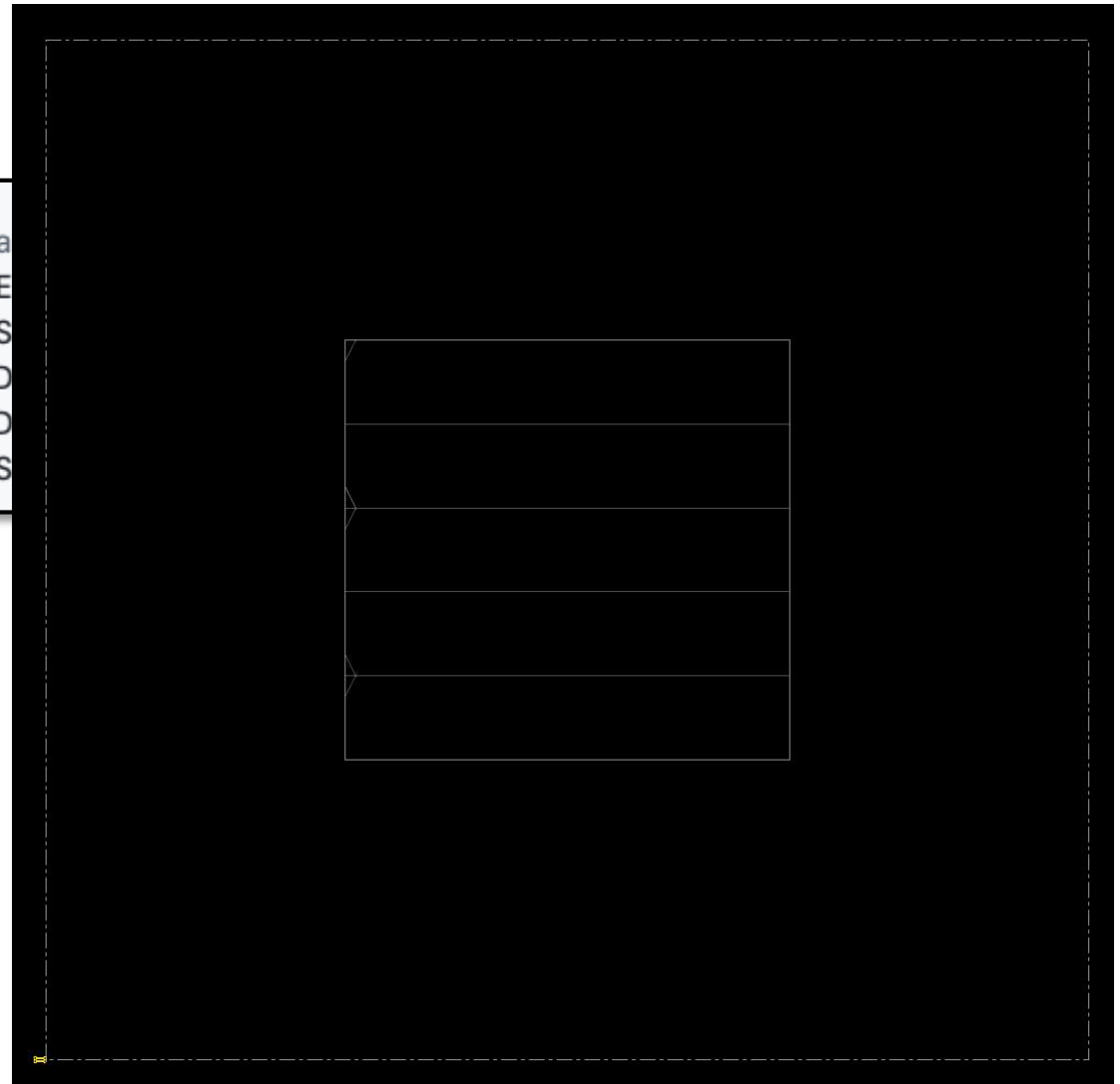
PnR steps – Initial Floor Planning

```
#Top-level module name
set DESIGN lfsr4
set VERILOG_FILES [list OUTPUTS/lfsr4_synth.v]
set SDC_FILE OUTPUTS/lfsr4_synth.sdc

# S
set # Plan how the overall layout, particula
set floorplan -r ${CORE_ASPECT_RATIO} ${CORE
set
set globalNetConnect VSS -type pgpin -pin VS
globalNetConnect VDD -type pgpin -pin VD
# S
# T globalNetConnect VDD -type tiehi -pin VD
set globalNetConnect VSS -type tielo -pin VS
set
set
set POWER_RING_OFFSET 0.5

# Setup PnR environment and say where library c
set init_verilog ${VERILOG_FILES}
set init_top_cell ${DESIGN}
set init_lef_file { \
    /usr/local/cadence/kits/tsmc/beLibs/65nm/TS
    /usr/local/cadence/kits/tsmc/beLibs/65nm/TS
    /usr/local/cadence/kits/tsmc/beLibs/65nm/TS
}
set init_mmmc_file "./mmmc_timing.tcl"
set init_gnd_net {VSS}
set init_pwr_net {VDD}

setDesignMode -process 65
init_design
```



PnR steps – Power ring, supply routing & well taps

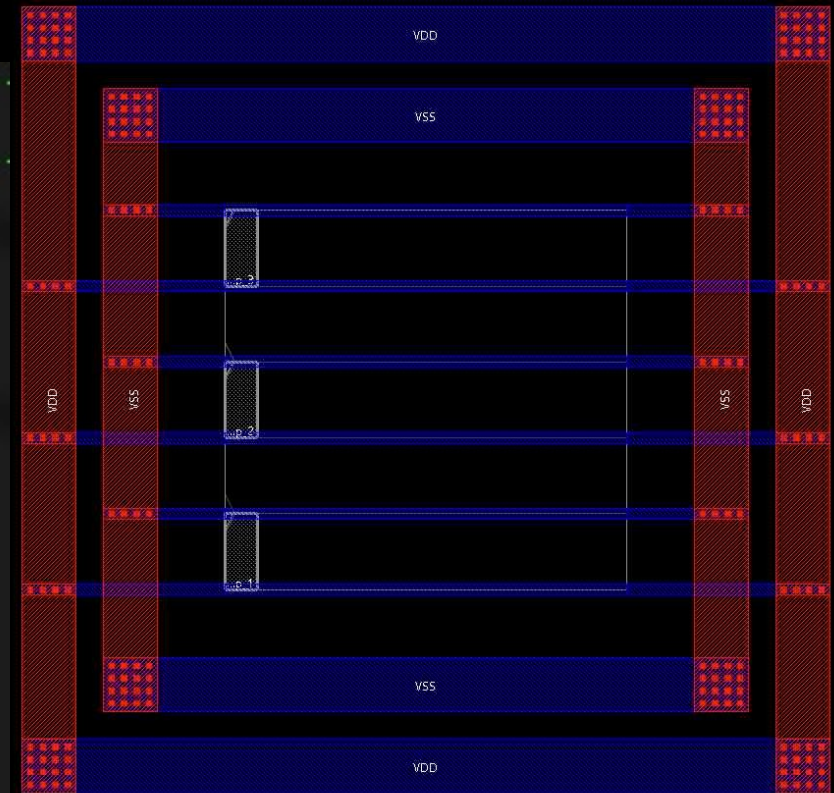
```
# Add power ring
addRing -width ${POWER_RING_WIDTH} -spacing ${POWER_RING_SPACING}
```

timeDesign Summary

Setup views included:
functional_typ

Setup mode	all	reg2reg	default
WNS (ns):	0.384	0.384	0.871
TNS (ns):	0.000	0.000	0.000
Violating Paths:	0	0	0
All Paths:	8	4	4
functional_typ	0.384	0.384	0.871
	0.000	0.000	0.000
	0	0	0
	8	4	4

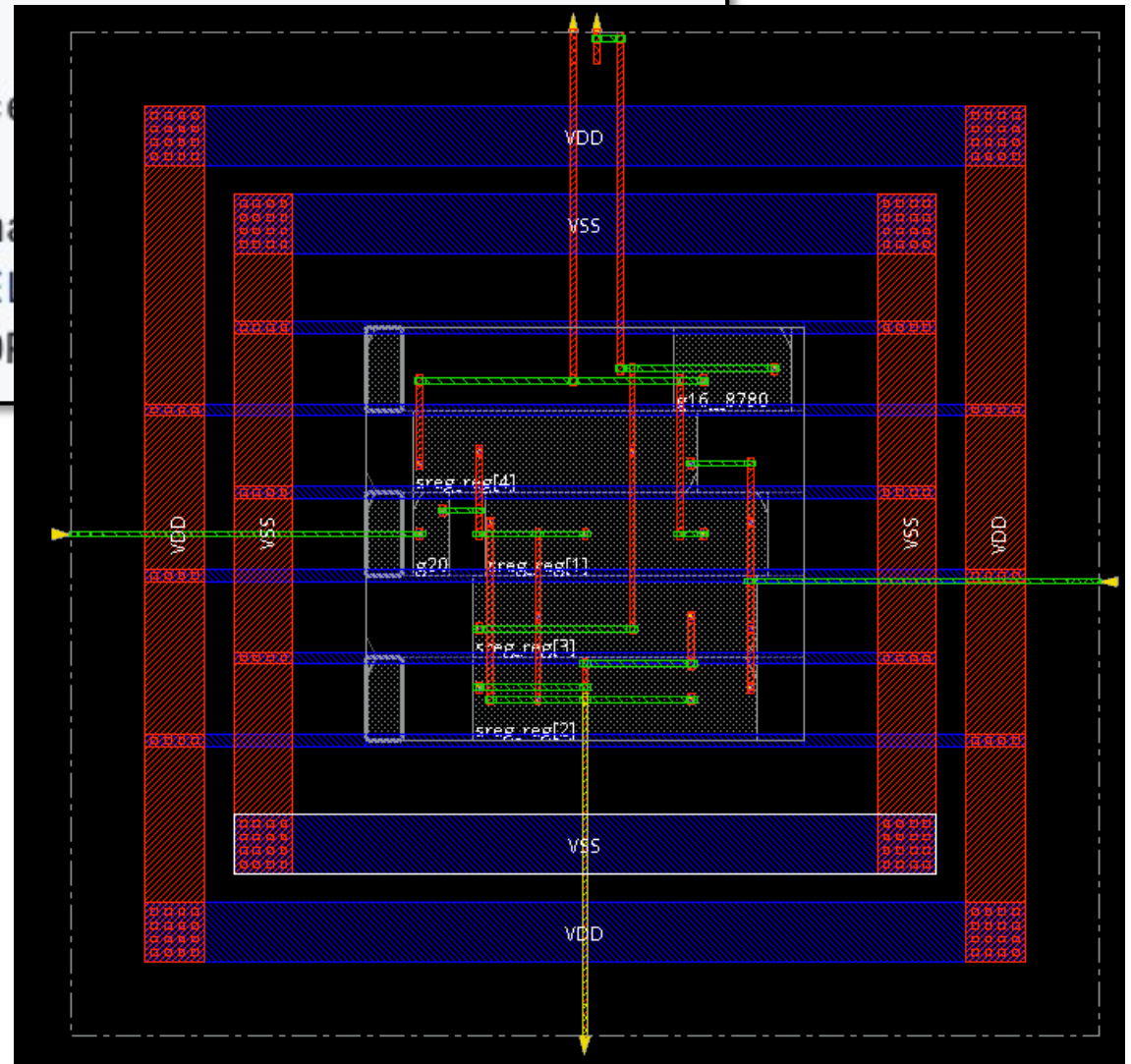
Density: 61.932%



```
/prePlace -prefix prePlace
```


PnR steps - Placement

```
# Placement
setPlaceMode -place_global_place
place_opt_design
setTieHiLoMode -maxFanout 10 -ma
addTieHiLo -cell "TIEHBWP7T TIE
timeDesign -preCTS -outDir REPO
```



PnR steps – Clock Tree Synthesis (CTS)

```
# specify clock properties
set_ccopt_property buffer_cells {CKBD0BWP7T CKBD1BWP7T CKBD2BWP7T CKBD3BWP7T}
set_ccopt_property inverter_cells {CKND0BWP7T CKND1BWP7T CKND2BWP7T CKND3BWP7T}
set_ccopt_property target_max_trans 130ps
set_ccopt_property target_skew 200ps
set_ccopt_property max_fanout 20

setOptMode -usefulSkew true
setOptMode -usefulSkewCCOpt extreme
```

```
# clock tree synthesis (CTS)
create_ccopt_clock_tree_spec -file REPORTS/ctsspec.tcl
source REPORTS/ctsspec.tcl
ccopt_design -check_prerequisites
ccopt_design
```

```
# post clock tree synthesis optimization
optDesign -postCTS -setup -hold -outDir REPORTS/postCTSOptTiming
timeDesign -postCTS -expandedViews -outDir REPORTS/postCTS -prefix postCTS
report_ccopt_clock_trees -file REPORTS/postCTS/clock_trees.rpt
report_ccopt_skew_groups -file REPORTS/postCTS/skew_groups.rpt
```


PnR steps – Routing & Finish up

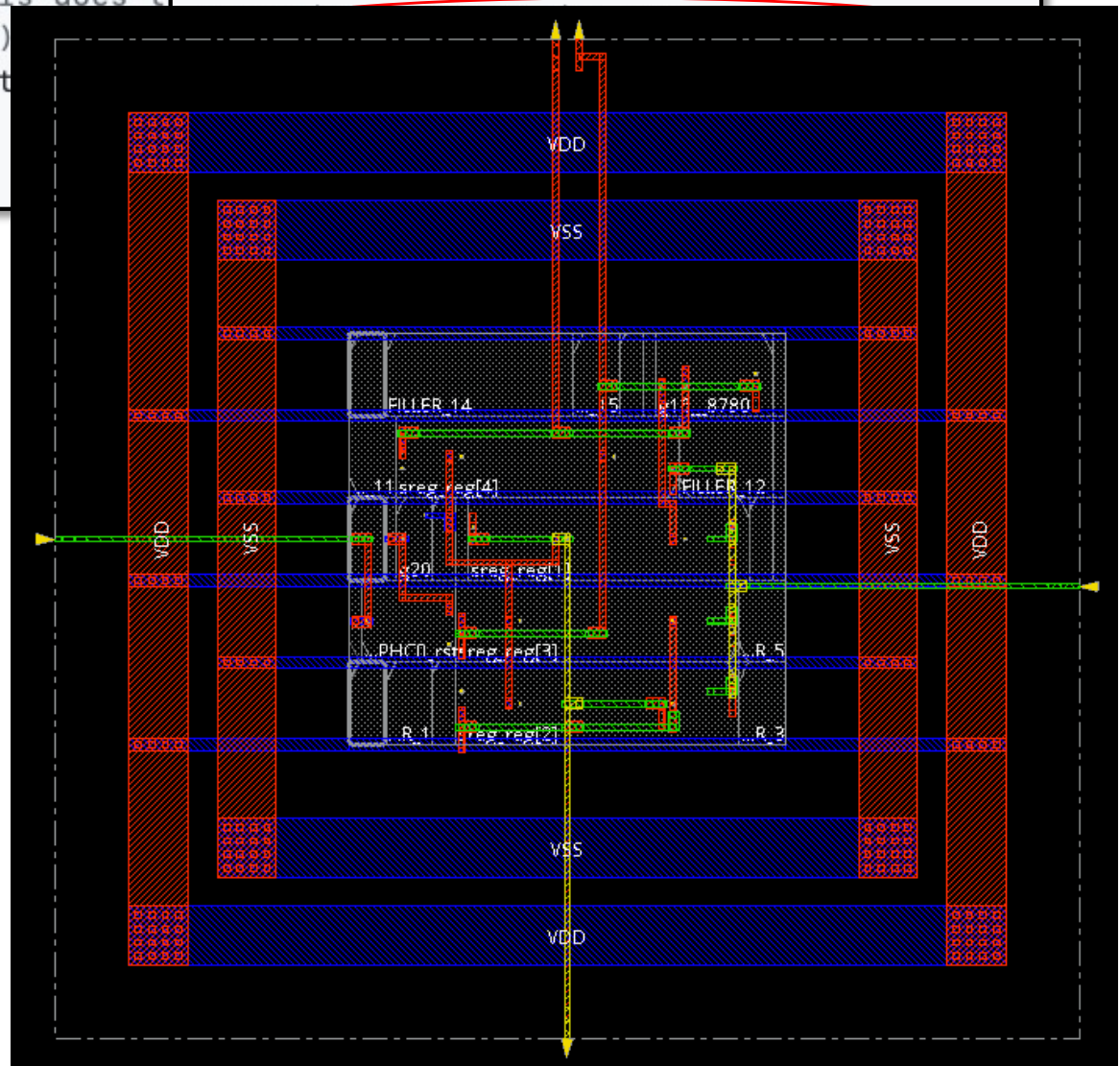
```
routeDesign                                ;# this does t
## Set analysis on on-chip variation (OCV)
setAnalysisMode -analysisType onChipVariat
optDesign -postRoute -setup
optDesign -postRoute -hold
```

lfsr4_soc.v

```
module lfsr4 (
    clk,
    rst,
    data_out);
    input clk;
    input rst;
    output [3:0] data_out;

    // Internal wires
    wire FE_PHN0_rst;
    wire n_0;
    wire n_1;

    DEL01BWP7T FE_PHC0_rst (.I(rst),
        .Z(FE_PHN0_rst));
    DFSNQD1BWP7T \sreg_reg[1] (.CP(clk),
        .D(n_1),
        .Q(data_out[0]),
        .SDN(n_0));
    CKX0R2D0BWP7T g16_8780 (.A1(data_out[2]),
        .A2(data_out[3]),
        .Z(n_1));
    DFCNQD1BWP7T \sreg_reg[4] (.CDN(n_0),
        .CP(clk),
        .D(data_out[2]),
        .Q(data_out[3]));
    DFCNQD1BWP7T \sreg_reg[3] (.CDN(n_0),
        .CP(clk),
        .D(data_out[1]),
        .Q(data_out[2]));
    DFCNQD1BWP7T \sreg_reg[2] (.CDN(n_0),
        .CP(clk),
        .D(data_out[0]),
        .Q(data_out[1]));
    INV0BWP7T g20 (.I(FE_PHN0_rst),
        .ZN(n_0));
endmodule
```



Simulation using Xcelium & GTKwave

Testbench (lfsr4_tb.sv)

```
`timescale 1ns/1ps
module lfsr4_tb;
  logic rst;
  logic clk;
  logic [3:0] data_out;

  // instantiate the DUT
  lfsr4 dut (
    .clk(clk),
    .rst(rst),
    .data_out(data_out)
  );

  initial begin
    $display("Start lfsr4 testbench ....");
    clk = 1'b0;
    rst = 1'b1;
    #1 clk = 1'b1;
    #1 clk = 1'b0;
    #1 rst = 1'b0;

    // clock generator
    for (int i=0; i<20; i++) begin
      #1 clk = 1'b1;
      #1 clk = 1'b0;
    end
  end

  initial begin
    $dumpfile("waveform.vcd");
    $dumpvars(1, lfsr4_tb, dut);
  end
endmodule
```

lfsr_soc.v



**Xcelium
(ncverilog)**

waveform.vcd



GTKwave

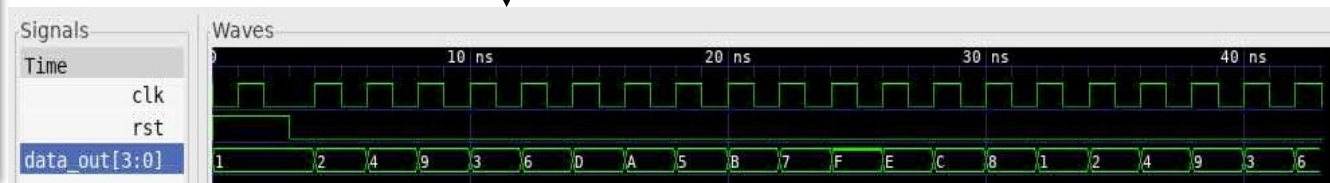
Makefile

```
# Directory definitions
LIB_DIR = "/usr/local/cadence"
SRC_DIR = ./
SDF_DIR = ./
WAVE_DIR = ./waves

# File definitions
TESTBENCH = SRC/lfsr4_tb.sv
DUT_SRCS = OUTPUTS/lfsr4_soc

# Run simulation with SDF and
run:
  ncverilog $(LIB_DIR)
  ncverilog -sv $(DUT_SRCS)

# View waveforms (assuming .
waves:
  gtkwave waveform.vcd &
```



Further Reading

- ◆ Publicly available standard cell library:

https://www.vlsitechnology.org/html/cells/vsclib013/lib_gif_index.html

- ◆ Genus Synthesis Flow Guide, v18.1 – stored on teaching servers

`/usr/local/cadence/GENUS_18.10.000/doc/genus_start/genus_start.pdf`

- ◆ Innovus User Guide, v18.1 – stored on teaching servers

`/usr/local/cadence/INNOVUS_18.10.000/doc/innovusUG/innovusUG.pdf`

- ◆ “New VLSI Tools”, Gajski&Kuhn ([10.1109/MC.1983.1654264](#))